

High-Level Programming with JavaScript, TypeScript, and React

Building a Todo App Progressively

Course Outline

1. **JavaScript** - Modern high-level features
2. **TypeScript** - Adding static types
3. **React** - Component-based architecture
4. **Comparison** - What we learned

We'll build the **same Todo app** three times, progressively!

Part 1: JavaScript

Modern High-Level Features

What Makes JavaScript "High-Level"?

- **Automatic memory management** (garbage collection)
- **Dynamic typing** (no need to declare types)
- **First-class functions** (functions as values)
- **Built-in high-order functions** (map, filter, reduce)
- **Object-oriented and functional programming support**
- **Asynchronous programming** (async/await, Promises)

Todo App: JavaScript Version

Goal: Create a simple todo app with:

- Add new tasks
- Mark tasks as complete
- Delete tasks
- Store in memory (array)

Let's explore the high-level features we use!

Feature 1: Arrow Functions

Traditional function:

```
function addTodo() {  
  // code here  
}
```

Arrow function (more concise):

```
const addTodo = () => {  
  // code here  
};
```

Benefits: Shorter syntax, lexical `this` binding

Feature 2: Template Literals

Old way (string concatenation):

```
li.innerHTML = '<span>' + todo.text + '</span>';
```

Template literals (easier to read):

```
li.innerHTML = `  
  <span>${todo.text}</span>  
  <button class="delete-btn">Delete</button>  
`;
```

Benefits: String interpolation, multi-line strings

Feature 3: Object Shorthand

Old way:

```
const newTodo = {  
  text: text,  
  completed: false,  
  id: Date.now()  
};
```

Object shorthand:

```
const newTodo = {  
  text, // same as text: text  
  completed: false,  
  id: Date.now()  
};
```

Feature 4: Spread Operator

Adding to array immutably:

```
// Instead of: todos.push(newTodo)  
todos = [...todos, newTodo];
```

Benefits: Creates new array, doesn't modify original

Why immutability? Easier to track changes, safer code

Feature 5: Array Methods (Functional Programming)

map - Transform each element:

```
todos = todos.map((todo, i) =>
  i === index ? { ...todo, completed: !todo.completed } : todo
);
```

filter - Remove elements:

```
todos = todos.filter((_, i) => i !== index);
```

forEach - Iterate over elements:

```
todos.forEach((todo, index) => {  
  // render each todo  
});
```

JavaScript Todo App: Complete Code

```
let todos = [];  
  
const addTodo = () => {  
  const input = document.getElementById('todoInput');  
  const text = input.value.trim();  
  
  if (text === '') return;  
  
  const newTodo = { text, completed: false, id: Date.now() };  
  todos = [...todos, newTodo];  
  
  input.value = '';  
  renderTodos();  
};
```

JavaScript: Pros and Cons

Pros:

-  Quick to write
-  Flexible and dynamic
-  No compilation step
-  Rich ecosystem

Cons:

-  No type checking (runtime errors)
-  Harder to maintain large codebases
-  IDE support limited

Part 2: TypeScript

Adding Static Types

What is TypeScript?

- **Superset of JavaScript** (all JS is valid TS)
- **Static type checking** at compile time
- **Better IDE support** (autocomplete, refactoring)
- **Compiles to JavaScript** (runs anywhere JS runs)

Key benefit: Catch errors before running code!

Feature 1: Interface Definitions

Define data structure:

```
interface Todo {  
  id: number;  
  text: string;  
  completed: boolean;  
}
```

Benefits:

- Clear documentation
- Type safety
- IDE autocomplete

Feature 2: Type Annotations

Variables with types:

```
let todos: Todo[] = [];
```

Function parameters and return types:

```
const addTodo = (): void => {  
  // code here  
};  
  
const toggleTodo = (index: number): void => {  
  // code here  
};
```

Feature 3: Type-Safe DOM Manipulation

JavaScript (unsafe):

```
const input = document.getElementById('todoInput');  
input.value = ''; // Could be null!
```

TypeScript (type assertion):

```
const input = document.getElementById('todoInput') as HTMLInputElement;  
input.value = ''; // TypeScript knows it's an input element
```

Note: `as` is an assertion, not a runtime check. If the element is missing, the code still fails at runtime.

Feature 4: Type-Safe Event Handling

```
input.addEventListener('keydown', (e: KeyboardEvent) => {  
  if (e.key === 'Enter') {  
    addTodo();  
  }  
});
```

Benefits:

- `e` has proper type
- IDE shows available properties
- Catch typos at compile time

Feature 5: Type Inference

TypeScript can **infer types** automatically (our `app.ts` uses this in `addTodo`, `toggleTodo`, `deleteTodo`):

```
const todos = [  
  { id: 1, text: "Learn TypeScript", completed: false }  
];  
// TypeScript knows todos is Array<{id: number, text: string, completed: boolean}>  
  
const result = todos.map(todo => todo.text);  
// TypeScript knows result is string[]
```

TypeScript Todo App: Key Differences

```
interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

let todos: Todo[] = [];

const addTodo = (): void => {
  const input = document.getElementById('todoInput') as HTMLInputElement;
  const text = input.value.trim(); // inferred as string

  if (text === '') return;

  const newTodo: Todo = { id: Date.now(), text, completed: false };
  todos = [...todos, newTodo];

  renderTodos();
};
```

TypeScript: Pros and Cons

Pros:

-  Type safety (catch errors early)
-  Better IDE support
-  Self-documenting code
-  Easier refactoring

Cons:

-  Learning curve
-  Compilation step required
-  More verbose code

Part 3: React

Component-Based Architecture

What is React?

- **JavaScript library** for building UIs
- **Component-based** architecture
- **Declarative** programming model
- **Virtual DOM** for efficient updates
- **One-way data flow** (props down, events up)

Key idea: Build UI from reusable components

React Core Concepts

1. **Components** - Reusable UI pieces
2. **Props** - Data passed to components
3. **State** - Component's internal data
4. **Hooks** - useState, useEffect, etc.
5. **JSX** - JavaScript + HTML syntax

Feature 1: Functional Components

```
const App: React.FC = () => {  
  return (  
    <div className="App">  
      <h1>Todo App</h1>  
    </div>  
  );  
};  
  
export default App;
```

Benefits:

- Encapsulated UI logic
- Reusable across app
- Easier to test

Feature 2: State Management with Hooks

useState Hook:

```
import { useState } from 'react';

const App: React.FC = () => {
  const [todos, setTodos] = useState<Todo[]>([]);

  const addTodo = (text: string) => {
    const newTodo = { id: Date.now(), text, completed: false };
    setTodos([...todos, newTodo]); // State update triggers re-render
  };

  return <div>...</div>;
};
```

Feature 3: Component Composition

Break UI into smaller components:

```
const App: React.FC = () => {  
  return (  
    <div className="container">  
      <h1>Todo App</h1>  
      <TodoInput onAdd={addTodo} />  
      <TodoList todos={todos} onToggle={toggleTodo} onDelete={deleteTodo} />  
    </div>  
  );  
};
```

Benefits: Separation of concerns, reusability

Feature 4: Props Interface

TodoInput Component:

```
interface TodoInputProps {
  onAdd: (text: string) => void;
}

const TodoInput: React.FC<TodoInputProps> = ({ onAdd }) => {
  const [text, setText] = useState<string>('');

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (text.trim()) {
      onAdd(text);
      setText('');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button type="submit">Add</button>
    </form>
  );
};
```

Feature 5: List Rendering

TodoList Component:

```
const TodoList: React.FC<TodoListProps> = ({ todos, onToggle, onDelete }) => {  
  return (  
    <ul>  
      {todos.map((todo) => (  
        <TodoItem  
          key={todo.id}      // Unique key for each item  
          todo={todo}  
          onToggle={onToggle}  
          onDelete={onDelete}  
        />  
      ))}  
    </ul>  
  );  
};
```

Key prop: Helps React identify which items changed

Feature 6: Controlled Components

Input controlled by React state:

```
const [text, setText] = useState('');

<input
  type="text"
  value={text} // Value from state
  onChange={(e) => setText(e.target.value)} // Update state
  placeholder="Enter a task..."
/>
```

Single source of truth: State drives the UI

Feature 7: Conditional Rendering

```
const TodoList: React.FC<TodoListProps> = ({ todos }) => {  
  return (  
    <ul>  
      {todos.map(todo => <TodoItem key={todo.id} todo={todo} />)}  
  
      {todos.length === 0 && (  
        <li className="empty-message">No tasks yet. Add one above!</li>  
      )}  
    </ul>  
  );  
};
```

React Project Structure

```
3-todo-react/  
├── App.tsx           # Main app component  
├── App.css           # Styles  
├── main.tsx          # Entry point  
├── index.html        # HTML template  
└── components/  
    ├── TodoInput.tsx # Input component  
    ├── TodoList.tsx  # List container  
    └── TodoItem.tsx  # Individual item
```

Separation: Each component in its own file

React: Pros and Cons

Pros:

-  Component reusability
-  Declarative UI
-  Large ecosystem
-  Excellent developer tools
-  Good for large applications

Cons:

-  Steeper learning curve
-  Build tools required
-  More boilerplate

Comparison

JavaScript vs TypeScript vs React

Same App, Different Approaches

Feature	JavaScript	TypeScript	React
Type Safety	No	Yes	Yes (with TS)
Build Step	No	Yes	Yes
Component Model	No	No	Yes
Code Organization	Functions	Functions + Types	Components
State Management	Manual	Manual	Built-in (Hooks)

Code Comparison: Adding a Todo

JavaScript:

```
const addTodo = () => {  
  const newTodo = { text, completed: false, id: Date.now() };  
  todos = [...todos, newTodo];  
  renderTodos();  
};
```

TypeScript:

```
const addTodo = (): void => {  
  const newTodo: Todo = { text, completed: false, id: Date.now() };  
  todos = [...todos, newTodo];  
  renderTodos();  
};
```

React:

```
const addTodo = (text: string): void => {  
  const newTodo: Todo = { text, completed: false, id: Date.now() };  
  setTodos([...todos, newTodo]); // Automatic re-render  
};
```

When to Use Each?

JavaScript:

- Small projects
- Quick prototypes
- Simple scripts
- Learning fundamentals

TypeScript:

- Medium to large projects
- Team collaboration
- Need type safety
- Complex business logic

React:

- Interactive UIs
- Large applications
- Reusable components
- Complex state management

High-Level Features Summary

JavaScript:

- Arrow functions, template literals, spread operator
- Object shorthand
- Array methods (map, filter, forEach)

TypeScript:

- All JavaScript features + Types
- Interfaces, type annotations
- Type inference

React:

- All TypeScript features + Components
- Hooks (useState, useEffect) and generics like `useState<Todo[]>`
- Props destructuring
- JSX, component composition
- Declarative UI

Key Takeaways

1. **High-level languages** provide abstraction from low-level details
2. **JavaScript** offers flexibility and quick development
3. **TypeScript** adds safety and maintainability
4. **React** provides structure and reusability
5. **Progressive enhancement**: Each layer builds on the previous
6. **Choose the right tool** for your project needs

Hands-On Exercise

Try all three versions:

1. Open `1-todo-javascript/index.html` - See plain JS in action
2. Compile and run `2-todo-typescript` - Experience type safety
3. Build and run `3-todo-react` - Explore component architecture

Compare:

- Code organization
- Developer experience
- Error handling
- Maintainability